Gesture-Controlled Live Video Effects and Image Processing Lab

Mini-Project Report

Course: Image and Video Processing

Submitted by

Lakshita Punekar (H013)

Yuvraj Singh Chouhan (H041)

Ganveer Singh Chouhan (H014)

Programme / Semester / Division: MBA Tech (CE) / 5TH / A

Faculty Name: Dr. RajGaurav Mishra

SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering

[MBA TECH CE]

Academic Year: 2026-27

Abstract

Problem addressed: Classical image-processing algorithms are usually taught with static code and hand-typed parameters, so it is hard to see how a parameter changes the result or how different methods compare on the same image. **Proposed solution:** Gesture IVP Lab is an interactive application in which a photo is captured with the webcam or uploaded, and syllabus operations are applied to it while the user controls everything with hand gestures recognised from the live webcam video. **Techniques used:** hand-landmark detection and tracking (MediaPipe Hands), temporal smoothing and hand-shape classification; grayscale conversion, point processing, histogram equalization, spatial filtering, binary and grayscale morphology, Prewitt, Sobel, LoG and Canny edge detection, and global, Otsu and adaptive thresholding (30 operations). **Outcome:** the application shows original and processed images side by side with the algorithm steps and histograms, runs at about 29 frames per second on an Intel Core i3 laptop, our Otsu implementation matches OpenCV exactly, and the median filter raises PSNR on salt-and-pepper noise from 17.37 dB to 28.55 dB.

Introduction

Background of the problem

A digital image is a two-dimensional array of intensities f(x, y). Image processing changes this array to enhance it, extract information or prepare it for analysis. The Image and Video Processing syllabus covers fundamentals, enhancement, transforms, morphology, edge detection, segmentation, colour processing and digital video. Each topic has parameters (a threshold T, a kernel size, a gamma value, a structuring element) whose effect is best understood by changing them and watching the result immediately. In practice students usually see only one before/after pair produced by a script.

Why the problem is relevant

- Seeing the output change while a parameter is varied, and comparing several methods on the same input, makes the concepts much easier to understand.
- Running each algorithm in a separate script is slow; one tool that holds many operations, explains them and compares them side by side shortens the learning loop.
- Touch-free (gesture) control is useful in classrooms, presentations, kiosks and hygienic places such as hospitals.

Role of image/video processing in solving it

Image and video processing is used in two places. **Video processing** drives the interface: every webcam frame is mirrored, down-scaled and converted to RGB, the hand is detected and tracked (21 landmarks), the landmarks are smoothed over time, and geometric tests turn them into hand shapes and actions. **Image processing** is the content of the lab: 30 syllabus operations are applied to the chosen photo, most of them written in NumPy so the algorithm is visible (Otsu, histogram equalization, erosion and dilation, Zhang-Suen thinning).

Motivation for selecting the project

Other projects in our class (Iron Man HUD, Sign Language to Speech, Finger Drawing, Air Piano) use hand tracking only as a controller. We wanted a project that also covers a large part of our syllabus in one working system, can be demonstrated live in the viva, and is interesting to use. Combining a gesture interface with a library of classical image-processing operations gave us both.

Problem Statement and Objectives

Which real-world problem are you solving?

- Classical image-processing operations are hard to understand from static images and code alone.
- There is no single, simple tool to apply many syllabus operations to one's own photo, tune their parameters live and compare methods side by side.
- Controlling a computer without touching it, using only a normal webcam, is useful for teaching, presentations and hygienic settings.

What are your objectives?

The objective of this project is to build an interactive, touch-free image-processing laboratory that recognises hand gestures from a live webcam video and uses them to select, tune and compare classical image-processing operations on a captured or uploaded photo, showing the original and processed images, the algorithm steps and quantitative values in real time.

Specific objectives

1. To detect the hand in live webcam video and recognise seven hand shapes (1, 2 and 3 fingers, pinch, open hand, thumbs up, fist), using smoothing and hold timers so one gesture gives exactly one action.
2. To implement 30 image-processing operations from the syllabus (enhancement, morphology, edge detection, segmentation), with the core algorithms written in NumPy.
3. To show Original vs Processed side by side with the algorithm steps, histograms, kernels and a Compare View of four related methods.
4. To change the parameter by pinching and moving the hand, and to rotate the image by tilting the open hand.
5. To keep the interface real-time (20+ frames per second) on a normal laptop.

Scope of the Project

What the project covers

- Image input from the webcam (3-2-1 countdown), from a file upload, or from a gallery of saved images.
- 30 operations in five categories: fundamentals, enhancement, morphology, edge detection, segmentation.
- Gesture control with seven hand shapes, with mouse and keyboard as a backup.
- Rotation of the image by hand tilt; every operation then works on the rotated image.
- Compare View, saving of results, and a browser-based interface (http://127.0.0.1:8000).

What it does not cover

- Frequency-domain filtering and transforms (DFT, DCT, Walsh-Hadamard), colour-model processing and video-coding topics (motion estimation, I/P/B frames) are not implemented in this version.
- No machine-learning model was trained by us; hand landmarks come from the pre-trained MediaPipe model.
- Gesture accuracy was not measured on a large labelled dataset of many users.
- The application runs locally on one computer; it is not an online service.

Type of images/videos used

Still images: colour photographs and test images (JPG, PNG, BMP, TIFF), processed at a maximum of 640 pixels on the longer side. Video: the live webcam stream (1280 x 720, about 30 frames per second) used for gesture recognition.

Application domain

Education (teaching image processing), touch-free human-computer interaction (presentations, kiosks, labs) and quick visual experimentation with classical operations.

Dataset / Input Description

Source of images/videos

- A synthetic test image generated with OpenCV and NumPy (images/sample_shapes.png).
- Photographs captured with the laptop webcam inside the app (captures/ folder).
- Photographs uploaded by the user from the laptop (images/ folder).
- Live video from the built-in webcam.

Dataset name, if applicable

Not applicable. The project is an interactive tool that works on any photo, so no public dataset was used.

Number of images/videos

Table 1. Number of inputs.

Input	Number
Synthetic test image (+ low-contrast variant derived from it)	1 (+1)
Webcam photographs captured during testing	3
Uploaded photographs	Any number (user's choice)
Live video	1 continuous webcam stream

Image resolution

Test image: 640 x 480 pixels. Webcam photographs: 1280 x 720 pixels. Uploaded photographs: any size. All photos are resized so the longer side is at most 640 pixels before processing (aspect ratio kept).

Image format

PNG for the test image and webcam captures; JPG, JPEG, PNG, BMP and TIFF are accepted for uploads. Images are 24-bit colour (8 bits per channel).

Video resolution and duration, if applicable

1280 x 720 pixels at about 30 frames per second (MJPG), continuous for as long as the app runs. For hand detection each frame is down-scaled to 640 x 360.

Sample input images/frames

The test image was designed for the syllabus operations: a ring with a hole, a rectangle with six small holes, a triangle with sharp corners, text with thin strokes, a speckled ellipse, a thin line, a colour gradient and Gaussian noise (sigma = 6).

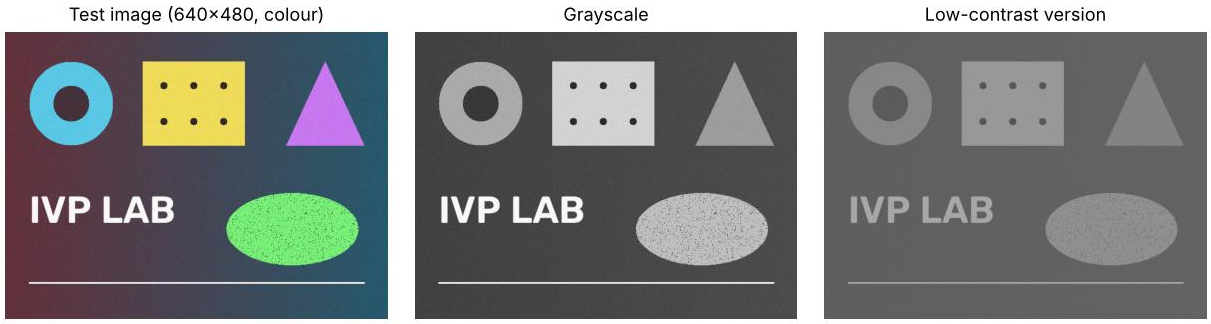


Figure 1. Sample inputs: test image, grayscale version and low-contrast version.

Methodology / Proposed Approach

Complete processing pipeline

1. **Frame capture:** a camera thread reads 1280 x 720 frames at 30 fps; the newest frame is mirrored.
2. **Hand detection:** the frame is down-scaled to 640 px and converted BGR to RGB; MediaPipe Hands returns 21 landmarks per hand.
3. **Smoothing:** landmarks are averaged over the last 5 frames to remove jitter.
4. **Gesture recognition:** finger states and hand shape are computed, debounced over 3 frames, and held shapes become events (next, previous, save, back, click, pinch, rotate).
5. **Image input:** a photo is captured or uploaded and resized to at most 640 px.
6. **Rotation:** the photo is rotated by the hand-tilt angle (affine warp).
7. **Processing:** the selected operation runs in a background thread (Otsu binarization first for morphology); results are cached.
8. **Display:** Original | Processed, the algorithm steps, histogram/kernel and values are drawn and streamed to the browser; a thumbs-up saves the result.

Block diagram / Flowchart

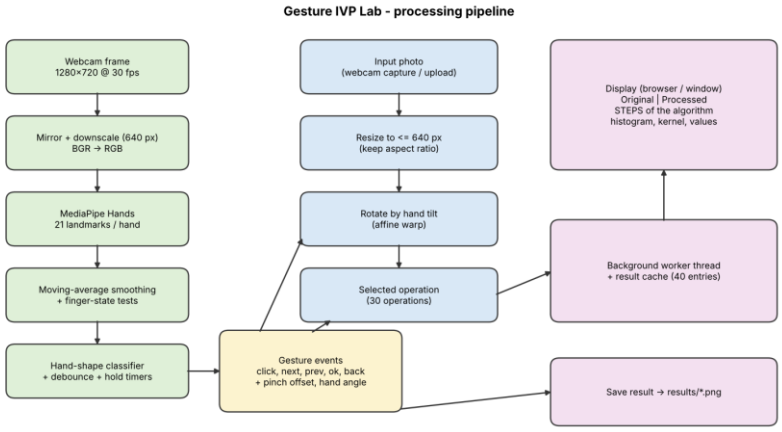


Figure 2. Block diagram. Green: gesture (video) path; blue: image path; yellow: gesture events; purple: processing back-end and output.

Algorithm pseudocode

```
FOR every webcam frame:
    frame = mirror(newest_frame); rgb = BGR2RGB(resize(frame, 640))
    landmarks = MediaPipeHands(rgb); smooth = mean(last 5 landmark sets)
    finger_up[i] = |wrist - tip_i| > 1.15 * |wrist - pip_i|
    shape = classify(finger_up, thumb, pinch distance)   # after 3 equal frames
    IF shape held long enough: emit event (next / prev / ok / back / click)
    IF pinch: parameter = start + (start_height - hand_height) / 0.35
    IF open hand: rotation = 1.5 * (angle - start_angle)  # 12 deg dead-band
ON change of photo / rotation / operation / parameter:
    img = rotate(resize(photo, 640), rotation)
    IF operation needs binary: img = otsu_threshold(gray(img))
    result = cache.get(key) or worker.run(operation, img, parameter); draw(result)
```

Techniques / Algorithms Used

Image Processing

Table 2. Image-processing techniques.

Technique	Used?	How it is used in the project
Grayscale conversion	Yes	Y = 0.299 R + 0.587 G + 0.114 B (own NumPy code).
Histogram equalization	Yes	Own code: s_k = round((CDF_k - CDF_min) / (N - CDF_min) x 255); before/after histograms.
Thresholding	Yes	s = 255 if r > T else 0; also negative, contrast stretching, level slicing, log, gamma (look-up tables).
Filtering	Yes	Mean (box), Gaussian, median (on salt & pepper noise), Laplacian sharpening, unsharp masking / high-boost.
Edge detection	Yes	Prewitt, Sobel $\|G\| = \sqrt{Gx^2 + Gy^2}$, Laplacian, Laplacian of Gaussian (zero-crossings), Canny.
Morphological operations	Yes	Own erosion/dilation (min/max over shifted images), opening, closing, boundary, hit-or-miss, Zhang-Suen thinning, thickening, grayscale top-hat/black-hat/gradient.
DFT/DCT	No	Not implemented in this version (future work).
Image segmentation	Yes	Global threshold; Otsu (own code maximising $w0 \cdot w1 \cdot (\mu0 - \mu1)^2$); adaptive mean/Gaussian.
Feature extraction	Yes	Hand features (finger extension, tilt angle, pinch distance); gradient magnitude; white-pixel %, mean, std.

Video Processing

Table 3. Video-processing techniques.

Technique	Used?	How it is used in the project
Frame extraction	Yes	Camera thread grabs frames continuously (1280 x 720, 30 fps); newest frame mirrored, down-scaled, BGR→RGB.
Background subtraction	No	Not needed: the hand is found by the MediaPipe palm detector.
Optical flow	No	Not used.
Motion detection	Partly	Hand movement from landmarks: vertical movement during a pinch, and 'pointer is still' (dwell) detection.
Object tracking	Yes	MediaPipe tracks the hand between frames; landmarks smoothed over 5 frames; same main hand kept.
Frame differencing	No	Not used in this version (planned for Live Video mode).

Implementation

Programming language

Python 3.11.9.

Software/tools

Visual Studio Code, Windows 11, Python virtual environment (venv), web browser (app shown at http://127.0.0.1:8000), Microsoft Word.

Libraries/frameworks

Python 3.11, OpenCV 4.11 (opencv-contrib-python), NumPy 1.26 and MediaPipe 0.10.14 (Hands) were used for implementation; Python's built-in http.server and threading for the browser interface; Matplotlib for the report graphs.

Hardware, if relevant

13th Gen Intel Core i3-1315U processor, 7.6 GB RAM, built-in HD webcam. No GPU was used.

Important implementation steps

1. **Project structure:** config.py (settings), gestures.py (gesture recognition), registry.py (list of operations), operations/ folder (one file per topic), ui.py (drawing), web.py (browser), main.py (app).
2. **Camera:** opened with Media Foundation (MSMF) and MJPG, giving 30 fps instead of 10 fps with DirectShow; automatic reconnection; only one copy of the app may run.
3. **Gesture engine:** finger tests use distances between landmarks, so tilted hands work. A first design with swipes and screen-edge zones was hard to use and was replaced by held hand shapes.
4. **Operation registry:** each operation is one function plus one @register(...) line with its parameter range and explanation; numbered docstring lines appear on screen as STEPS.
5. **Speed:** hand detection runs in its own thread and image operations in a worker thread with a 40-entry result cache; static parts of the screen are drawn once.
6. **Browser interface:** frames are JPEG-encoded and streamed (MJPEG); clicks, keys and uploads are sent back by HTTP; gesture guide cards light up for the detected hand shape.

Results and Discussion

Input vs. output images

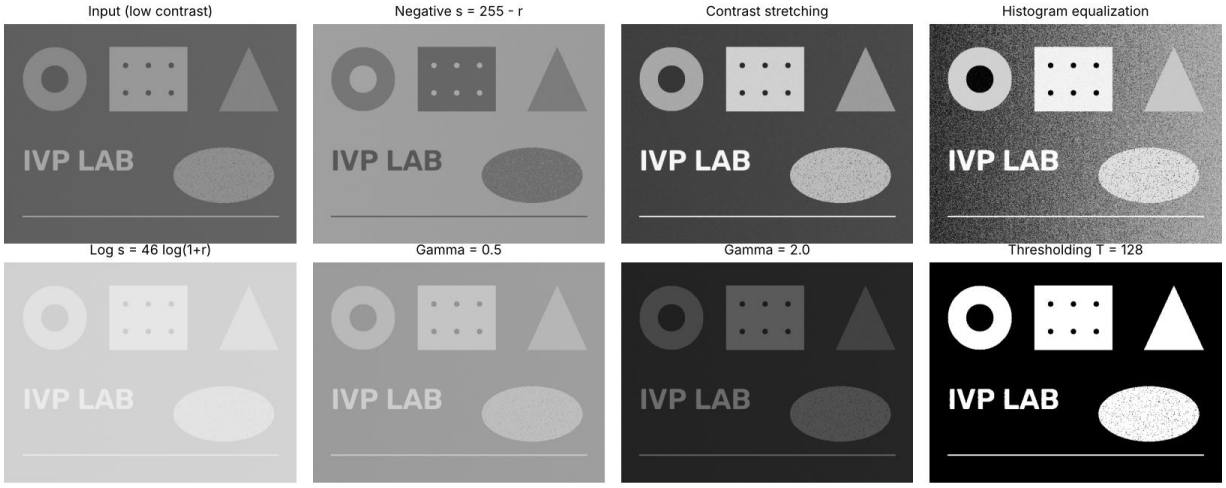


Figure 3. Point processing on the low-contrast input, and thresholding of the original.

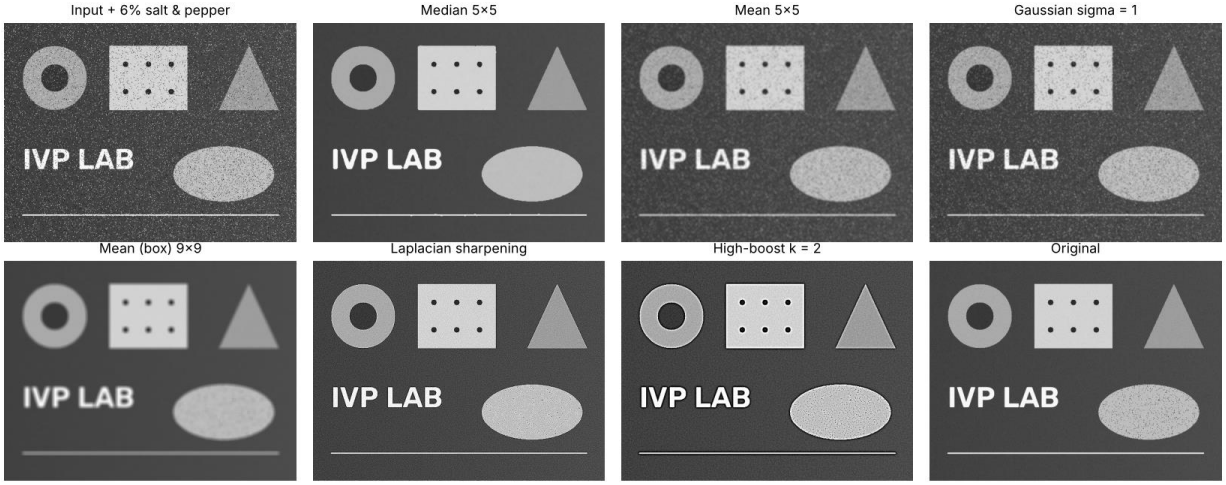


Figure 4. Spatial filtering: salt-and-pepper noise removed by median and mean filters; smoothing, sharpening, high-boost.

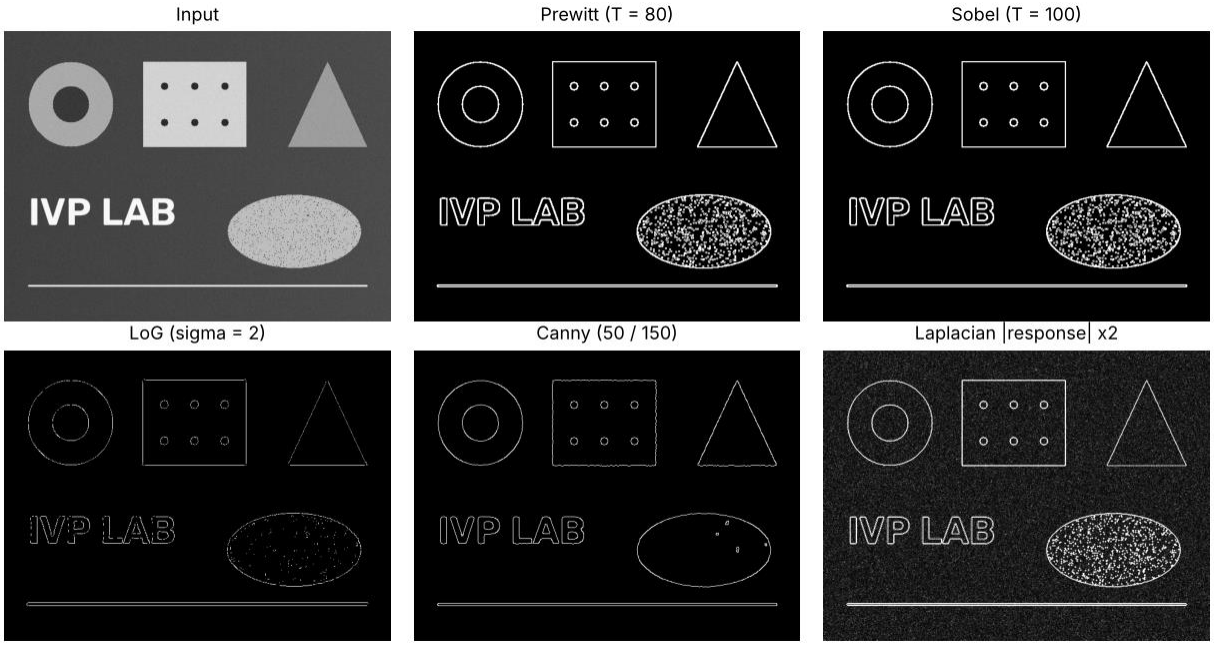


Figure 5. Edge detectors on the same input.

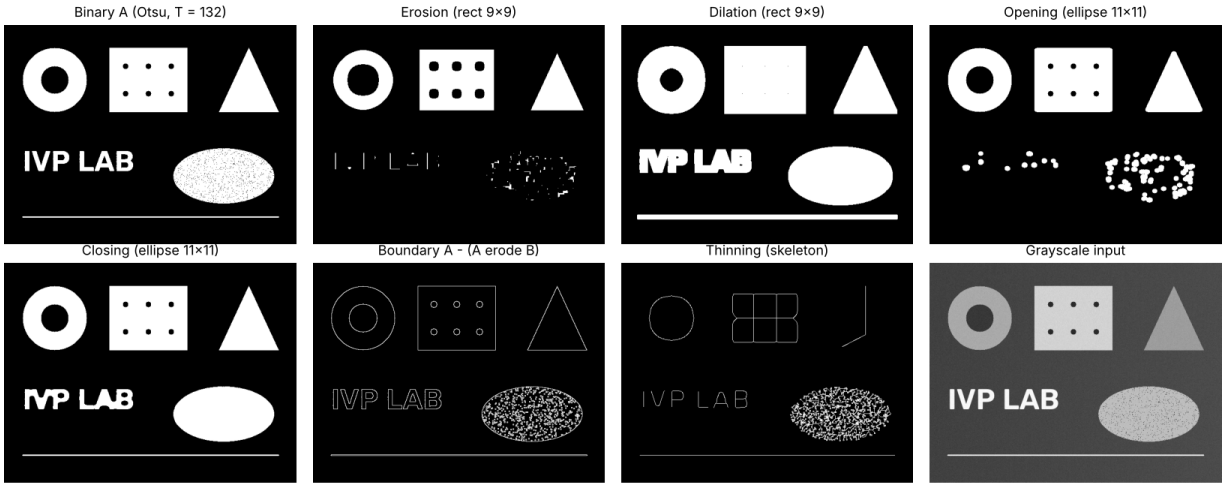


Figure 6. Binary morphology on the Otsu-binarized test image.

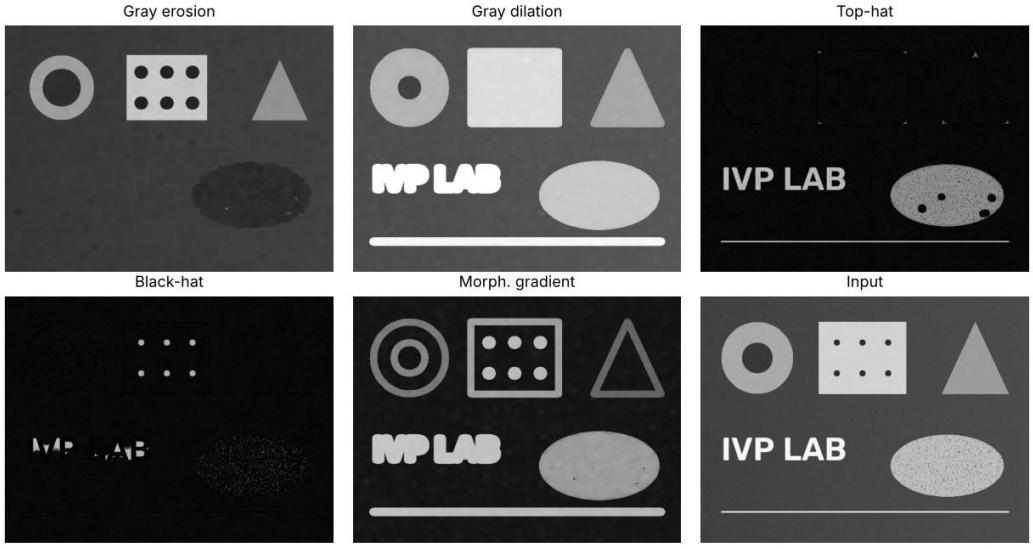


Figure 7. Grayscale morphology with a 15 x 15 elliptical structuring element.

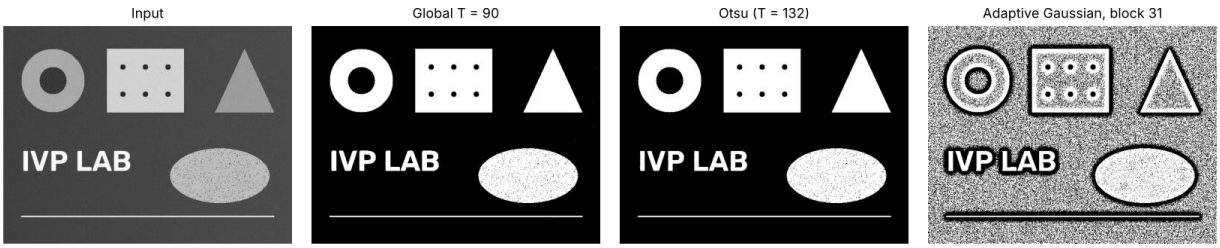


Figure 8. Segmentation: global, Otsu and adaptive thresholding.

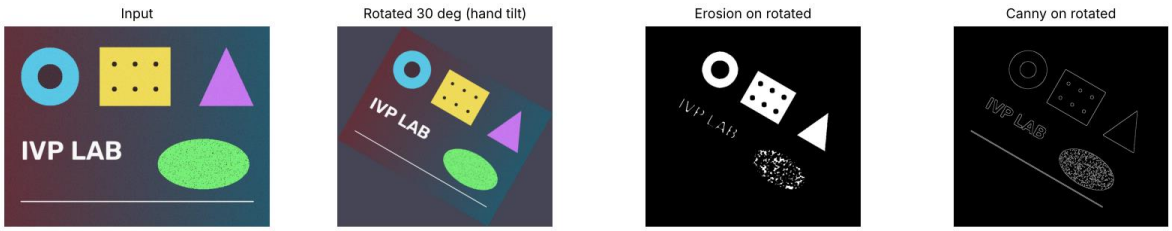


Figure 9. Image rotated by 30 degrees (open-hand tilt) and operations applied to it.

Intermediate processing results

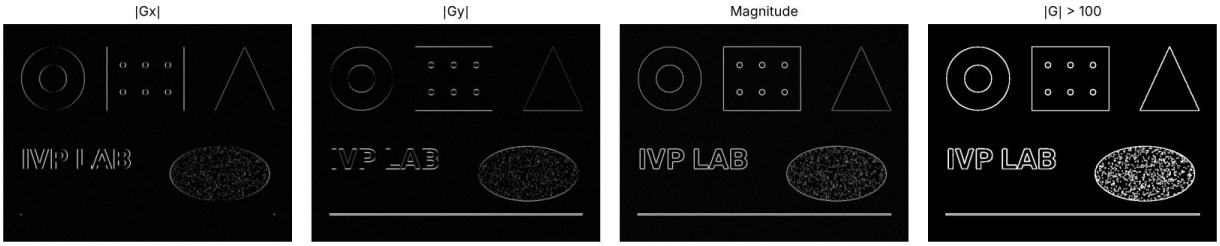
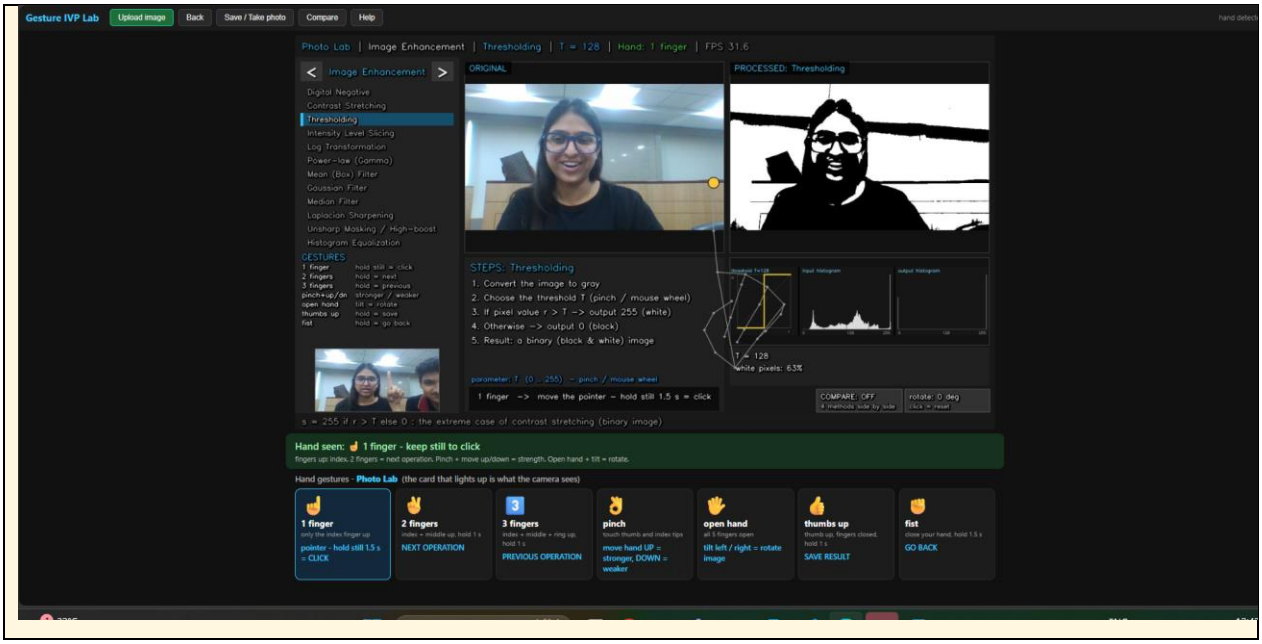
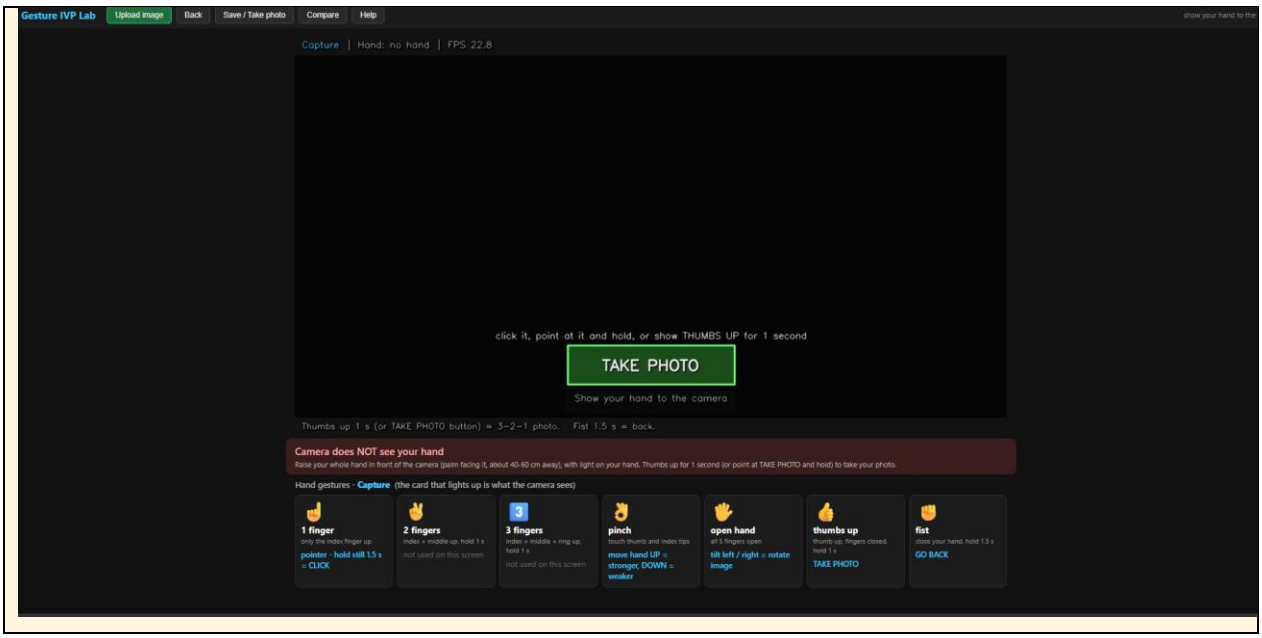


Figure 10. Intermediate results of the Sobel operator: |Gx|, |Gy|, gradient magnitude and thresholded edges.

For morphology the photo is first binarized with Otsu's threshold; for the median filter, salt-and-pepper noise is added first so the effect is visible; for LoG, the smoothed Laplacian response is computed before zero-crossings are extracted. The app shows these intermediate images next to the result.

Screenshots





To take a screenshot on Windows press Win + Shift + S, select the app window, then click inside the box above and press Ctrl + V. Delete the yellow box afterwards.

Graphs/charts

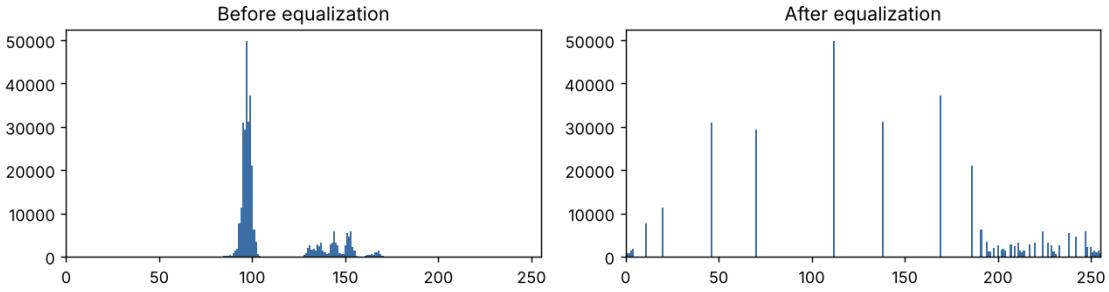


Figure 11. Histogram of the low-contrast input before and after histogram equalization.

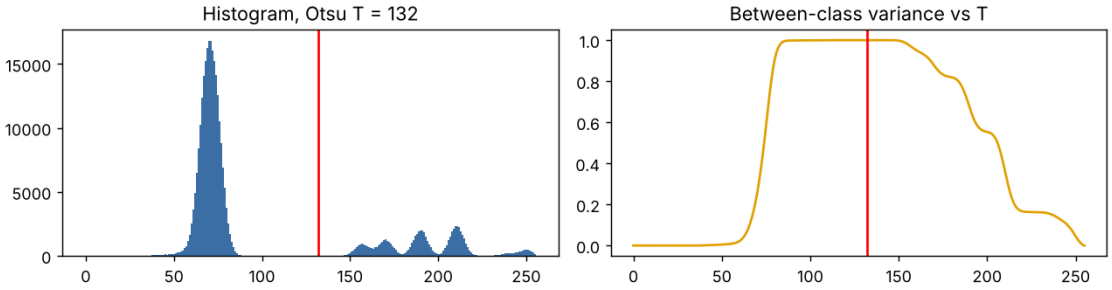


Figure 12. Otsu's method: histogram with the chosen threshold, and between-class variance vs T.

Tables

Table 4. Area change produced by morphology.

Operation	White pixels (%)	Effect seen
Binary input A	23.5	-
Erosion (rect 9x9)	12.1	Objects shrink; thin line and text strokes removed; holes grow
Dilation (rect 9x9)	30.5	Objects grow; the six small holes are filled
Opening (ellipse 11x11)	17.2	Thin parts removed; large shapes keep their size
Closing (ellipse 11x11)	24.0	Gaps in the speckled ellipse closed

Boundary A - (A erode B)	3.4	1-pixel outlines

Table 5. Edge detectors compared.

Detector	Edge pixels (%)	Observation
Prewitt	5.37	Thick (2-3 px) edges, some noise
Sobel	5.35	Similar to Prewitt, slightly smoother
Laplacian of Gaussian	1.35	Thin closed contours
Canny	1.63	Thin, continuous edges, least noise

Table 6. Our Otsu implementation vs OpenCV.

Test image	Our Otsu (NumPy)	OpenCV Otsu
Test image	132	132
Low-contrast	121	121
Equalized	112	112
Gaussian-blurred	127	127

Quantitative performance measures

Table 7. Denoising quality, PSNR = 10 log10(255^2 / MSE).

Image (vs clean grayscale)	PSNR (dB)
Noisy input (6% salt & pepper)	17.37
Mean filter 5x5	24.1
Gaussian filter, sigma = 1	24.67
Median filter 5x5	28.55

Table 8. Before / after measurements.

Measure	Before	After
Histogram equalization: gray-level range	82-170	0-255
Histogram equalization: standard deviation	21.1	70.2
Otsu threshold after rotation (black corners vs border fill)	111	131
Interface speed (detection inline vs own thread)	8.1 fps	28.8-30.2 fps
Camera capture (DirectShow vs MSMF + MJPG)	10.0 fps	30.3 fps

Table 9. Gesture and functional tests.

Test	Result
Gesture detection rate (background thread)	18.6-23.8 fps
Synthetic hand-shape classification tests	9 / 9 correct
Synthetic hold / event timing tests	10 / 10 correct (no early or double firing)
Automated app tests (menus, rotation, pinch, compare, save, all 30 operations)	All passed

Discussion. The median filter gives the best denoising (28.55 dB) because the 0 and 255 outliers are never the median of a neighbourhood, while mean and Gaussian filters only spread them out. Histogram equalization raised the contrast about 3.3 times but also amplified background noise. Canny produced the thinnest and cleanest edges. Our Otsu implementation matched OpenCV on every image. Filling rotated corners with the border colour kept Otsu's threshold close to the original, whereas black corners pulled it down.

Challenges / Limitations

Problems encountered

- The first gesture design (fast swipes, a control mode, palm at the screen edge) was hard to use; it was replaced by hand shapes that are held still.
- Several copies of the app competed for the webcam and showed black images; solved with a single-instance lock and automatic reconnection.
- The default DirectShow camera driver gave only 10 fps; MSMF + MJPG gave 30 fps.

Difficult cases

- A bright window or wall behind the user darkens the hand and makes detection harder.
- Hands partly outside the frame, too far (over 1 m) or too close are not detected.
- A relaxed thumb can make '1 finger' look like a pinch; a loose fist can look like thumbs up.

Computational limitations

- MediaPipe needs about 50-65 ms per frame on the Core i3 CPU (no GPU).
- Zhang-Suen thinning takes about 0.6-1.8 s per image, so its result appears slightly after the pinch.
- The webcam takes about 8 seconds to start.

Dataset limitations

- No public labelled dataset was used; tests used a synthetic image, a few webcam photos and synthetic landmark sequences.
- Gesture accuracy across many users, skin tones, backgrounds and lighting was not measured.

Cases where the method failed

- Rotating the image with black corners lowered Otsu's threshold (fixed by filling corners with the border colour).
- Global thresholding with a fixed T fails on the gradient background; adaptive thresholding also outlines texture.
- Histogram equalization amplifies noise in flat regions.

Conclusion

What was implemented?

Gesture IVP Lab: a touch-free image-processing laboratory that captures or uploads a photo and applies 30 syllabus operations to it under hand-gesture control, showing Original vs Processed, the algorithm steps, graphs and values, with a Compare View and saving of results.

What techniques were used?

Video processing: frame capture, hand detection and tracking (MediaPipe Hands), temporal smoothing, finger features, shape classification and hold timers. Image processing: grayscale conversion, point processing, histogram equalization, spatial filtering, morphology, edge detection, thresholding-based segmentation and affine rotation.

What was achieved?

All five objectives were met: seven gestures, 30 operations, side-by-side comparison with steps, pinch and rotation control, and about 29 fps. The measured results match theory: median filtering gave 28.55 dB PSNR, equalization raised contrast from 21.1 to 70.2, and our Otsu matched OpenCV exactly.

What is your learning?

We learned how classical algorithms work at the pixel level (look-up tables, convolution, order statistics, structuring elements, between-class variance) and how their parameters change the output. We also learned that real-time video analysis needs smoothing, debouncing and multithreading to feel reliable, and that simple gestures with clear feedback work better than complex ones. Future work: frequency-domain filters, DCT/DFT, colour processing and video motion estimation.

References

[1] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[2] W. Burger and M. J. Burge, Digital Image Processing: An Algorithmic Introduction, 2nd ed., Springer, 2022.

[3] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62-66, 1979.

[4] J. Canny, "A computational approach to edge detection," IEEE Trans. PAMI, vol. 8, no. 6, pp. 679-698, 1986.

[5] D. Marr and E. Hildreth, "Theory of edge detection," Proc. Royal Society of London B, vol. 207, pp. 187-217, 1980.

[6] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," Communications of the ACM, vol. 27, no. 3, pp. 236-239, 1984.

[7] J. Serra, Image Analysis and Mathematical Morphology, Academic Press, 1982.

[8] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

[9] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[10] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[11] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[12] Python Software Foundation, Python 3.11 documentation, https://docs.python.org/3.11/